

Résumé Projet IN213

CaioSHell — Interpréteur de shell en OCaml

Caio ESCORCIO LIMA DOURADO
ENSTA Paris

25 Mai 2026

Table des matières

1	Vue d'ensemble	3
1.1	Architecture	3
1.2	Pipeline d'exécution	3
1.3	Inspiration générale	4
1.4	Typage	4
2	Valeurs et environnement	4
2.1	Le type <code>value</code>	4
2.2	L'environnement σ	5
2.3	Extension de l'environnement	5
3	Sémantique opérationnelle	5
3.1	Jugement d'évaluation	5
3.2	Règles d'inférence	5
4	Règles d'évaluation	6
4.1	Littéraux	6
4.2	<code>EIdent</code> et <code>EVar</code>	6
4.3	<code>ESeq</code>	6
4.4	<code>EDefVar</code> et <code>EAssign</code>	6
4.5	<code>EDefFun</code>	7
4.6	<code>EApp</code>	7
4.7	<code>EBinop</code>	7
4.8	<code>EIf</code> et <code>EAnd</code>	8
4.9	<code>EWhile</code>	8
4.10	<code>EFor</code>	8
5	Fonctions utilitaires	8
5.1	Réimplémentation sans le module <code>List</code>	8
5.2	<code>assoc</code> et <code>assoc_opt</code> — recherche dans l'environnement	9
5.3	<code>fold_left</code> — propagation de l'environnement	9
5.4	<code>fold_left2</code> — liaison des paramètres	9
5.5	<code>to_array</code> — conversion pour <code>execvp</code>	10
6	Commandes Unix	10
6.1	Vue générale de <code>eval_cmd</code>	10
6.2	Commandes internes (<i>builtins</i>)	10
6.3	Exécution externe : <code>fork</code> + <code>execvp</code>	10

7	Pipes Unix	11
7.1	Principe	11
7.2	Protocole	11
8	Résumé	12

1 Vue d'ensemble

Objectif : concevoir un interpréteur d'un langage shell personnalisé (`.csh`), exécuté directement via les appels système Unix, sans compilation préalable.

1.1 Architecture

Fichier	Rôle
<code>cshlex.mll</code>	Analyse lexicale (tokens)
<code>cshparse.mly</code>	Analyse syntaxique (AST)
<code>cshast.ml</code>	Définition des nœuds AST
<code>cshinterp.ml</code>	Interpréteur / évaluateur
<code>cshloop.ml</code>	Boucle REPL interactive
<code>test.csh</code>	Exemples de la syntaxe du code

1.2 Pipeline d'exécution

```
source .csh
  V
cshlex.mll
  V
tokens
  V
cshparse.mly
  V
AST
  V
cshinterp.ml
  V
fork / execvp
  V
resultat Unix
```

Exemple

```
ls -la /tmp;;
```

— Lexer :

```
IDENT PATH PATH SEMISEMI
```

— Parser :

```
ECmd[
  EIdent "ls";
  EIdent "-la";
  EIdent "/tmp"
]
```

— Interpréteur :

```
fork + execvp "ls"
[|"ls"; "-la"; "/tmp"|]
```

1.3 Inspiration générale

L'implémentation de `cshinterp.ml` s'appuie sur deux sources d'inspiration principales.

foo.ml — modèle pour les appels système Unix. Le fichier `foo.ml` fourni dans le cours définit deux fonctions, `exec_command` et `pipe_commands`, qui illustrent l'utilisation de `Unix.fork`, `Unix.execvp`, `Unix.pipe` et `Unix.waitpid`. Les fonctions `eval_cmd` et `eval_pipe` reprennent directement cette structure : séparation fils/père, nommage des PIDs (`child_pid`, `dead_pid`), flag `Unix.WUNTRACED` et gestion des codes retour.

Module List d'OCaml — fonctions réimplémentées. L'interpréteur *n'importe pas* le module `List`. Les fonctions nécessaires (`map`, `fold_left`, `fold_left2`, `iter`, `assoc`, `assoc_opt`) sont réécrites à la main, ce qui rend leur rôle explicite dans le contexte de l'interpréteur. De même, `to_array` remplace `Array.of_list` sans faire appel à la bibliothèque standard.

1.4 Typage

Le langage possède un **typage dynamique** : aucune vérification statique n'est effectuée avant l'exécution.

Les erreurs de types apparaissent uniquement au moment où les valeurs sont réellement utilisées.

2 Valeurs et environnement

2.1 Le type value

```
type value =  
  | VInt    of int  
  | VBool   of bool  
  | VString of string  
  | VUnit  
  | VClosure of (string list * Cshast.expr * env)
```

Constructeur	Exemple
VInt 42	42, @x + 1
VBool true	booléens, codes retour Unix
VString "abc"	chaînes, chemins, commandes
VUnit	absence de valeur significative
VClosure	fonction utilisateur

Closures

Une `VClosure` capture :

1. les paramètres,
2. le corps AST,
3. l'environnement σ_f au moment de la définition.

Cela implémente la **portée lexicale**.

2.2 L'environnement σ

```
type env = (string * value) list
```

L'environnement est une liste d'association :

$$\sigma = [(\text{"x"}, \text{VInt } 10); (\text{"name"}, \text{VString "Alice"})]$$

2.3 Extension de l'environnement

```
let bind x v env = (x,v) :: env
```

Notation sémantique :

$$\sigma[x \mapsto v]$$

Masquage (shadowing)

$$\sigma = [(x, 5)] \quad \sigma[x \mapsto 10] = [(x, 10); (x, 5)]$$

La recherche retourne toujours la liaison la plus récente. Un ancien lien n'est jamais supprimé — il est simplement masqué.

3 Sémantique opérationnelle

3.1 Jugement d'évaluation

$$\langle e, \sigma \rangle \rightarrow \langle v, \sigma' \rangle$$

Lecture :

« L'expression e , évaluée dans l'environnement σ , produit la valeur v et le nouvel environnement σ' . »

Correspondance directe avec OCaml :

```
val eval :  
  Cshast.expr -> env -> value * env
```

3.2 Règles d'inférence

Forme générale :

$$\frac{\text{prémisses}}{\text{conclusion}}$$

Une règle sans prémisse est un axiome.

4 Règles d'évaluation

4.1 Littéraux

$$\langle \text{EInt } n, \sigma \rangle \rightarrow \langle \text{VInt } n, \sigma \rangle \quad \langle \text{EBool } b, \sigma \rangle \rightarrow \langle \text{VBool } b, \sigma \rangle$$

$$\langle \text{EString } s, \sigma \rangle \rightarrow \langle \text{VString } s, \sigma \rangle \quad \langle \text{EPass}, \sigma \rangle \rightarrow \langle \text{VUnit}, \sigma \rangle$$

Les littéraux ne modifient jamais l'environnement. **EPass** est un no-op syntaxique utilisé dans les branches vides.

4.2 EIdent et EVar

Identifiant libre (EIdent) :

$$\frac{s \in \text{dom}(\sigma)}{\langle \text{EIdent } s, \sigma \rangle \rightarrow \langle \sigma(s), \sigma \rangle} \quad \frac{s \notin \text{dom}(\sigma)}{\langle \text{EIdent } s, \sigma \rangle \rightarrow \langle \text{VString } s, \sigma \rangle}$$

Interprétation

Si l'identifiant n'existe pas dans σ , il devient automatiquement une chaîne de caractères. Cela permet d'écrire `ls /tmp` sans guillemets : `"ls"` et `"/tmp"` sont absents de σ et deviennent des **VString** passés à `execvp`.

Variable stricte (EVar, syntaxe @x) :

$$\frac{s \in \text{dom}(\sigma)}{\langle \text{EVar } s, \sigma \rangle \rightarrow \langle \sigma(s), \sigma \rangle} \quad s \notin \text{dom}(\sigma) \Rightarrow \text{Unbound variable}$$

L'absence d'une variable `@x` est une erreur de programmation, contrairement à un identifiant libre.

4.3 ESeq

$$\frac{\langle e_1, \sigma \rangle \rightarrow \langle v_1, \sigma_1 \rangle \quad \dots \quad \langle e_n, \sigma_{n-1} \rangle \rightarrow \langle v_n, \sigma_n \rangle}{\langle \text{ESeq}[e_1; \dots; e_n], \sigma \rangle \rightarrow \langle v_n, \sigma_n \rangle}$$

L'environnement est **propagé de gauche à droite** : une définition en position i est visible aux positions $i + 1 \dots n$. Implémenté par `fold_left` :

```
let step (_, acc_env) e = eval e acc_env in
fold_left step (VUnit, env) es
```

4.4 EDefVar et EAssign

$$\frac{\langle e, \sigma \rangle \rightarrow \langle v, \sigma' \rangle}{\langle \text{EDefVar}(x, e), \sigma \rangle \rightarrow \langle \text{VUnit}, \sigma'[x \mapsto v] \rangle}$$

Observation

EDefVar (`define var x = e`) et **EAssign** (`x = e`) utilisent exactement la même règle sémantique : la différence est **uniquement syntaxique**.

4.5 EDefFun

$$\langle \text{EDefFun}(f, \text{params}, \text{body}), \sigma \rangle \rightarrow \langle \text{VUnit}, \sigma[f \mapsto \text{VClosure}(\text{params}, \text{body}, \sigma)] \rangle$$

Portée lexicale

La fermeture capture l'environnement σ **au moment de la définition**, pas au moment de l'appel. Si une variable est redéfinie après **define function**, la fonction utilise toujours la valeur capturée.

4.6 EApp

$$\frac{\sigma(f) = \text{VClosure}(ps, \text{body}, \sigma_f) \quad \forall i : \langle a_i, \sigma \rangle \rightarrow \langle v_i, _ \rangle \quad \langle \text{body}, \sigma_c \rangle \rightarrow \langle v, _ \rangle}{\langle \text{EApp}(f, \text{args}), \sigma \rangle \rightarrow \langle v, \sigma \rangle}$$

```
| Cshast.EApp (f, args) ->
  (match lookup f env with
  | VClosure (params, body, closure_env) ->
    let eval_arg a = fst (eval a env) in
    let bind_param new_env param value = bind param value new_env in
    let values      = map eval_arg args in
    let call_env =
      try fold_left2 bind_param closure_env params values
      with Invalid_argument _ ->
        failwith (f ^ ": wrong number of arguments")
    in
    let (v, _) = eval body call_env in
    (v, env)    (* l'env de l'appelant est inchangé *)
  | _ -> failwith (f ^ " is not a function"))
```

Points clés

- Les arguments sont évalués dans l'env de **l'appelant**.
- Le corps s'exécute dans σ_f (env capturé) étendu des paramètres.
- Les modifications du corps sont **locales** : σ de l'appelant est retourné inchangé.
- `fold_left2` associe chaque paramètre à sa valeur ; si les listes diffèrent en taille, `Invalid_argument` est levé.

4.7 EBinop

$$\frac{\langle e_1, \sigma \rangle \rightarrow \langle v_1, \sigma_1 \rangle \quad \langle e_2, \sigma_1 \rangle \rightarrow \langle v_2, \sigma_2 \rangle}{\langle \text{EBinop}(op, e_1, e_2), \sigma \rangle \rightarrow \langle op(v_1, v_2), \sigma_2 \rangle}$$

Opérateur	Types	Résultat
+	int × int	somme entière
+	str × str/int	concaténation
-, *	int × int	entier
#	int × int	division entière
=	tout type	booléen structural
> < >= <=	int ou str	booléen

Pourquoi # pour la division ?

Le caractère / est réservé par le lexeur aux chemins de fichiers (token PATH). La division entière utilise donc # : $10 \# 3 \rightarrow \text{VInt } 3$.

4.8 EIf et EAnd

EIf :

$$\frac{\langle \text{cond}, \sigma \rangle \rightarrow \langle v, \sigma' \rangle \quad \text{truthy}(v) \quad \langle t, \sigma' \rangle \rightarrow \langle v_t, \sigma_t \rangle}{\langle \text{EIf}(\text{cond}, t, f), \sigma \rangle \rightarrow \langle v_t, \sigma_t \rangle}$$

$$\frac{\langle \text{cond}, \sigma \rangle \rightarrow \langle v, \sigma' \rangle \quad \neg \text{truthy}(v) \quad \langle f, \sigma' \rangle \rightarrow \langle v_f, \sigma_f \rangle}{\langle \text{EIf}(\text{cond}, t, f), \sigma \rangle \rightarrow \langle v_f, \sigma_f \rangle}$$

EAnd (court-circuit) :

$$\frac{\langle e_1, \sigma \rangle \rightarrow \langle v_1, \sigma_1 \rangle \quad \text{truthy}(v_1) \quad \langle e_2, \sigma_1 \rangle \rightarrow \langle v_2, \sigma_2 \rangle}{\langle \text{EAnd}(e_1, e_2), \sigma \rangle \rightarrow \langle v_2, \sigma_2 \rangle}$$

$$\frac{\langle e_1, \sigma \rangle \rightarrow \langle v_1, \sigma_1 \rangle \quad \neg \text{truthy}(v_1)}{\langle \text{EAnd}(e_1, e_2), \sigma \rangle \rightarrow \langle \text{VBool false}, \sigma_1 \rangle}$$

Fonction `is_truth` : $\text{VBool } b \rightarrow b$, $\text{VInt } n \rightarrow n \neq 0$, $\text{VString } s \rightarrow s \neq ""$, $\text{VUnit} \rightarrow \text{false}$.

4.9 EWhile

$$\frac{\langle \text{cond}, \sigma \rangle \rightarrow \langle v, \sigma' \rangle \quad \neg \text{truthy}(v)}{\langle \text{EWhile}(\text{cond}, \text{body}), \sigma \rangle \rightarrow \langle \text{VUnit}, \sigma' \rangle} [\text{STOP}]$$

$$\frac{\langle \text{cond}, \sigma \rangle \rightarrow \langle v, \sigma' \rangle \quad \text{truthy}(v) \quad \langle \text{body}, \sigma' \rangle \rightarrow \langle _, \sigma'' \rangle \quad \langle \text{EWhile}(_, \sigma''), \sigma'' \rangle \rightarrow \langle u, \sigma''' \rangle}{\langle \text{EWhile}(\text{cond}, \text{body}), \sigma \rangle \rightarrow \langle u, \sigma''' \rangle} [\text{ITER}]$$

Une référence mutable `env_ref` transporte σ entre les itérations, de sorte qu'une affectation dans le corps (ex. `i = @i + 1`) soit visible lors de la prochaine évaluation de la condition.

4.10 EFor

$$\frac{\langle e_i, \sigma \rangle \rightarrow \langle v_i, _ \rangle \quad \langle \text{body}, \sigma[x \mapsto v_i] \rangle \rightarrow \langle _, \sigma_i \rangle}{\text{itération sur } e_i : \sigma \leftarrow \sigma_i}$$

À chaque itération, la variable de boucle x est liée à la valeur courante dans une extension de `env_ref`. Le corps est réévalué pour chaque élément de la liste.

5 Fonctions utilitaires

5.1 Réimplémentation sans le module List

Par choix pédagogique, `cshinterp.ml` n'importe aucun module de la bibliothèque standard OCaml (`List`, `Array`, *etc.*). Les fonctions suivantes sont réécrites à la main :

Fonction	Équivalent stdlib	Rôle	Utilisée dans
----------	-------------------	------	---------------

<code>assoc</code>	<code>List.assoc</code>	Recherche par clé, lève <code>Not_found</code>	<code>lookup</code>
<code>assoc_opt</code>	<code>List.assoc_opt</code>	Recherche par clé, retourne option	<code>EIdent</code>
<code>map</code>	<code>List.map</code>	Applique f à chaque élément	<code>EApp</code> , <code>eval_cmd</code>
<code>fold_left</code>	<code>List.fold_left</code>	Réduction séquentielle gauche→droite	<code>ESeq</code>
<code>fold_left2</code>	<code>List.fold_left2</code>	Réduction parallèle sur deux listes	<code>EApp</code>
<code>iter</code>	<code>List.iter</code>	Parcours avec effets de bord	<code>EFor</code>
<code>to_array</code>	<code>Array.of_list</code>	Conversion liste → tableau	<code>eval_cmd</code>

5.2 `assoc` et `assoc_opt` — recherche dans l’environnement

La recherche est linéaire et retourne la **première** (donc la plus récente) liaison trouvée. C’est ce qui implémente le masquage (*shadowing*).

```
let rec assoc key = function
| []                -> raise Not_found
| (k, v) :: _ when k = key -> v
| _ :: rest         -> assoc key rest
```

`assoc_opt` retourne `None` au lieu de lever une exception. Elle est utilisée par `EIdent` pour le repli sur `VString` : si un nom n’est pas lié dans σ , il est traité comme une chaîne littérale (utile pour les noms de commandes comme `ls`, `grep`, etc.).

5.3 `fold_left` — propagation de l’environnement

```
let rec fold_left f acc = function
| []      -> acc
| x :: xs -> fold_left f (f acc x) xs
```

Dans `ESeq`, l’accumulateur est la paire (`valeur`, `env`). À chaque étape, la valeur précédente est ignorée (`_`) et le nouvel environnement est transmis à l’expression suivante :

```
let step (_, acc_env) e = eval e acc_env in
fold_left step (VUnit, env) es
```

5.4 `fold_left2` — liaison des paramètres

```
let rec fold_left2 f acc l1 l2 =
  match l1, l2 with
  | [], []      -> acc
  | x :: xs, y :: ys -> fold_left2 f (f acc x y) xs ys
  | _          -> raise (Invalid_argument "fold_left2")
```

Dans `EApp`, elle lie chaque paramètre à sa valeur dans `closure_env` :

```
let bind_param new_env param value = bind param value new_env in
fold_left2 bind_param closure_env params values
```

Si les deux listes ne sont pas de même longueur, `Invalid_argument` est capturée et convertie en "wrong number of arguments".

5.5 to_array — conversion pour execvp

`Unix.execvp` attend un `string array`, mais les arguments sont construits comme une liste. `to_array` effectue la conversion sans `Array.of_list` :

```
let to_array lst = match lst with
| [] -> [|]
| hd :: _ ->
  let rec len acc = function
    | [] -> acc
    | _ :: t -> len (acc+1) t in
  let arr = Array.make (len 0 lst) hd in
  let rec fill i = function
    | [] -> ()
    | x :: xs -> arr.(i) <- x ; fill (i+1) xs
  in fill 0 lst ; arr
```

6 Commandes Unix

6.1 Vue générale de eval_cmd

Inspirée de `exec_command` dans `foo.ml`, `eval_cmd` suit trois étapes :

1. **Évaluation des arguments** : chaque expression est évaluée puis convertie en chaîne via `value_to_string`.
2. **Filtrage des commandes internes** (*builtins*) : `cd` et `exit` sont traitées directement dans le processus shell.
3. **Exécution externe** : toute autre commande est lancée via `fork` + `execvp`.

```
let arg_to_str a = value_to_string (fst (eval a env)) in
let strs = map arg_to_str args in
```

6.2 Commandes internes (builtins)

Commande	Raison d'être interne
<code>cd</code>	<code>Unix.chdir</code> dans un processus fils ne change que le répertoire de ce fils, pas celui du shell courant. La commande doit donc s'exécuter dans le processus shell lui-même.
<code>exit</code>	Doit terminer l'interpréteur lui-même. Un <code>exit</code> dans un fils ne terminerait que le fils.

6.3 Exécution externe : fork + execvp

Structure directement inspirée de `exec_command` dans `foo.ml` :

```
match Unix.fork () with
| 0 ->
  (* Fils : remplace son image par la commande a executer.
   Le premier argument de argv[] est le nom de la commande. *)
  Unix.execvp cmd (to_array (cmd :: rest))
```

```

| child_pid ->
  (* Pere : attend explicitement la fin de ce fils. *)
  match Unix.waitpid [Unix.WUNTRACED] child_pid with
  | (_, Unix.WEXITED 0)          -> (VBool true,  env)
  | (_, Unix.WEXITED _)         -> (VBool false, env)
  | (dead_pid, Unix.WSIGNALED s) ->
    Printf.eprintf "process %d interrupted by signal %d\n" dead_pid s;
    (VBool false, env)
  | (aslept_pid, Unix.WSTOPPED s) ->
    Printf.eprintf "process %d stopped by signal %d\n" aslept_pid s;
    (VBool false, env)

```

Code retour et VBool

Un code retour 0 correspond à `VBool true`; tout autre code correspond à `VBool false`. Ce comportement est **intentionnel** : des commandes comme `test` retournent 1 pour signifier « faux » — ce n’est pas une erreur. Le flag `Unix.WUNTRACED` (repris de `foo.ml`) permet de détecter également les processus arrêtés (`Ctrl+Z`).

7 Pipes Unix

7.1 Principe

Inspirée de `pipe_commands` dans `foo.ml`, `eval_pipe` relie deux expressions par un tube Unix :

$$\frac{\langle e_1, \sigma \rangle \rightarrow \langle _, _ \rangle \quad \langle e_2, \sigma \rangle \rightarrow \langle v, \sigma' \rangle}{\langle \text{EPipe}(e_1, e_2), \sigma \rangle \rightarrow \langle v, \sigma' \rangle}$$

7.2 Protocole

1. `Unix.pipe()` crée la paire `(fd_in, fd_out)`.
2. `stdin` est sauvegardé avec `Unix.dup(stdin)`.
3. **Fils gauche** : redirige `stdout` vers `fd_out`, évalue e_1 , puis quitte (`exit 0`).
4. **Père** : redirige `stdin` depuis `fd_in`, évalue e_2 .
5. Le père attend la fin du fils (`waitpid`), puis restaure `stdin` original.

```

let (fd_in, fd_out) = Unix.pipe () in
let saved_stdin = Unix.dup Unix.stdin in
match Unix.fork () with
| 0 ->
  (* Fils gauche : écrit dans le pipe *)
  Unix.dup2 fd_out Unix.stdout;
  Unix.close fd_out; Unix.close fd_in; Unix.close saved_stdin;
  ignore (eval e1 env); exit 0
| left_pid ->
  (* Père : lit depuis le pipe *)
  Unix.dup2 fd_in Unix.stdin;
  Unix.close fd_out; Unix.close fd_in;
  let result = eval e2 env in
  ignore (Unix.waitpid [Unix.WUNTRACED] left_pid);
  Unix.dup2 saved_stdin Unix.stdin;
  Unix.close saved_stdin;
  result

```

Différence avec `foo.ml`

Dans `foo.ml`, le pipe est définitif : le processus père n'a plus besoin de lire depuis le clavier après le pipe. Dans `CaioShell`, le père est le REPL lui-même. Après l'exécution du pipe, il doit continuer à lire les commandes de l'utilisateur. `Unix.dup(stdin)` sauvegarde le descripteur original, et `Unix.dup2(saved_stdin, stdin)` le restaure une fois le pipe terminé.

8 Résumé

Construction	Étend σ	Visible à l'appelant
<code>EDefVar</code> , <code>EAssign</code>	oui	oui
<code>EDefFun</code>	oui	oui
<code>EApp</code> (corps)	local	non
<code>ESeq</code>	propagation	oui
<code>EWhile</code> , <code>EFor</code>	propagation	oui
<code>EIf</code> , <code>EAnd</code>	branche exécutée	oui
<code>ECmd</code> , <code>EPipe</code>	non	non
littéraux, <code>EPass</code>	non	non

Conclusion

Le modèle sémantique :

$$\langle e, \sigma \rangle \rightarrow \langle v, \sigma' \rangle$$

reflète directement la signature OCaml :

```
eval : expr -> env -> value * env
```

Le projet combine :

- interprétation dynamique sans phase de typage,
- portée lexicale via les fermetures (`VClosure`),
- appels système Unix (`fork`, `execvp`, `pipe`),
- propagation fonctionnelle de l'environnement,
- fonctions utilitaires réimplémentées sans le module `List`.

Exemple de code testé

```
#>> CaioShell - Tests

#>> Simple commands
echo " 1. Simple commands ";;
echo "Hello, CaioShell!";;
pwd;;

#>> Variables
echo " Variables ";;
define var name = "Alice";;
echo @name;;
define var n = 42;;
echo @n;;

#>> Arithmetic
```

```

echo " Arithmetic ";;
define var a = 10;;
define var b = 3;;
define var sum = @a + @b;;
define var diff = @a - @b;;
define var prod = @a * @b;;
define var quot = @a # @b;;
echo @sum;;
echo @diff;;
echo @prod;;
echo @quot;;

#>> String concatenation
echo " String concatenation ";;
define var s1 = "Hello, ";;
define var s2 = "World!";;
define var msg = @s1 + @s2;;
echo @msg;;

#>> Function definition and call
echo " Functions ";;
define function greet(person) {
    echo @person;
};;
greet("Alice");;
greet("Bob");;

#>> Function with arithmetic
echo " Function with arithmetic ";;
define function double(x) {
    define var result = @x * 2;
    echo @result;
};;
double(5);;
double(21);;

#>> Pipes
echo " Pipes ";;
echo "hello world" | wc -w;;
ls / | grep tmp;;

#>> And (&&)
echo " And (&&) ";;
(echo "first") && (echo "second");;

#>> If / else
echo "If / else ";;
if (test 1 "-eq" 1) {
    echo "condition true";
} else {
    echo "condition false";
};;
if (test 1 "-eq" 2) {
    echo "should not print";
} else {
    echo "correctly false";
};;

```

```

#>> For loop
echo " For loop ";;
for (@item in ["alpha", "beta", "gamma"]) {
    echo @item;
};;

#>> While loop (avec entiers negatifs)
echo " While loop ";;
define var i = 0;;
while (test @i "-gt" -3) {
    echo @i;
    i = @i - 1;
};;

#>> Variable reassignment
echo " Variable reassignment ";;
define var counter = 100;;
counter = @counter + 1;;
echo @counter;;

#>> Pass
echo " Pass ";;
if (test 0 "-eq" 1) {
    echo "unreachable";
} else {
    pass;
};;
echo "pass ok";;

#>> Files with dots in name
echo "File paths with dots ";;
cat /etc/hostname;;

#>> Flags (dash args)
echo " Command flags ";;
ls -l /tmp;;

#>> cd
echo "cd ";;
cd /tmp;;
pwd;;
cd /;;
pwd;;

echo " All tests done ";;

```